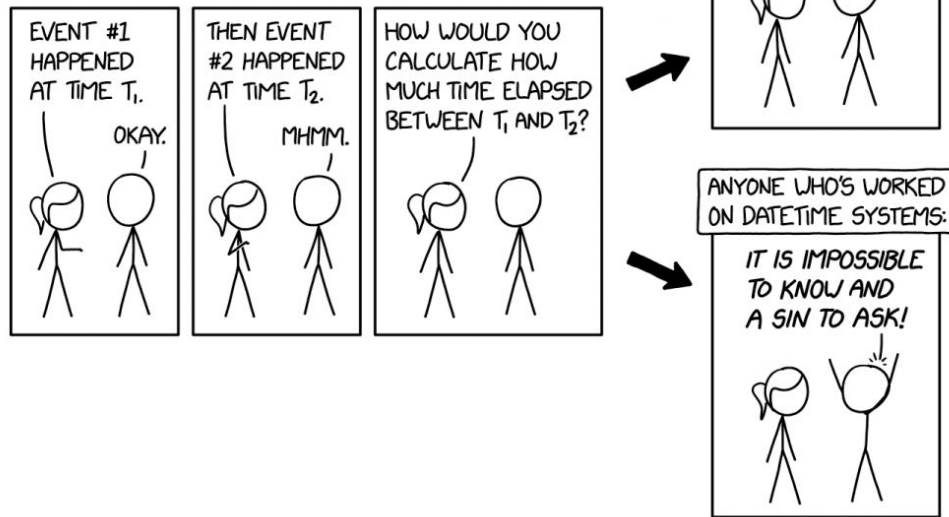




Software Engineering

Causality & Distributed Systems

Multi-Process With Users

So far:

Multi-process for performance reasons

Other uses too:

- Geographically spaced backups
- Improved latency for users

Multi-Process With Users

User 1, connected to Server 1, changes some state at time t_1

Ex: picking up an item off the ground in a game

User 2, connected to Server 2, wants to change the same state at time t_2

What should happen?

Clock Drift

By themselves, two servers can't provide a consistent ordering of events

- in practice, clocks drift constantly

How to order events anyway?

Central Time Server

Use NTP (Network Time Protocol) to sync

Most common:

`pool.ntp.org` (or filtered versions)

`time.nist.gov`

This reduces clock drift into the millisecond range (typically 200-500ms), but doesn't completely eliminate it

Central Time Server

Sticking with the central server approach:

How to handle conflicting data due to (small, unusual) clock drift?

Central Time Server

Sticking with the central server approach:

How to keep all the servers in sync about the state of the world?

Logical Clocks

What if you don't have a central server?

- Geographically remote locations
- Air-gapped networks for security reasons
- Intermittent connectivity issues
- Very high latency connections

More common than you'd think (ex: Taiwan)

Logical Clocks

Any **monotonically increasing** sequence can be used as a clock

12:01 pm

Or

t_1

12: 05 pm

t_2

1:10 pm

t_3

Logical Clocks

Each server maintains a "logical clock" - the time only increases when something happens

- Each server knows what order things happened on for that server

What problem does this cause?

Vector Clocks

Instead of one clock, keep a list of clocks. Now a "timestamp" looks like:

[server 1= t_1 , server 2 = t_2]

For server 1, its entry is based on its logical clock

Every other entry is based on the "last time the server synced"

Vector Clocks

What do things "happening at the same time" look like in this system?

Vector Clocks

Common application: git

Instead of servers/logical clocks, git knows about branches and commits

One commit = one action

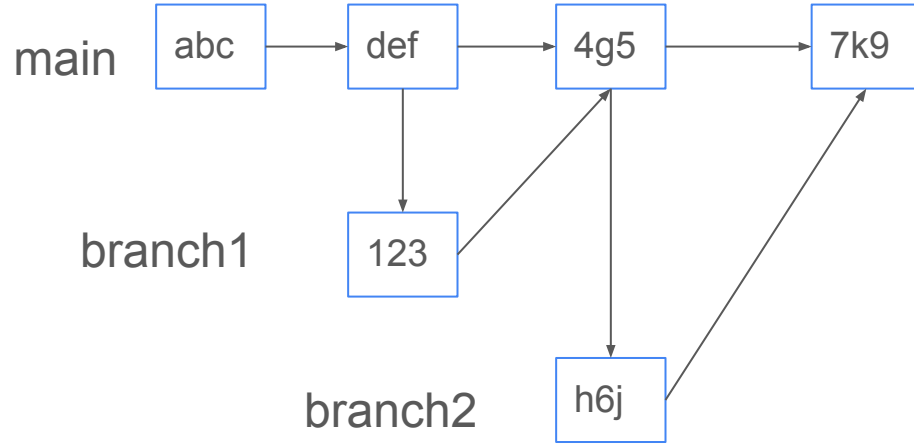
Commits are applied as **diffs**

Changes happening "at the same time" = merge conflict

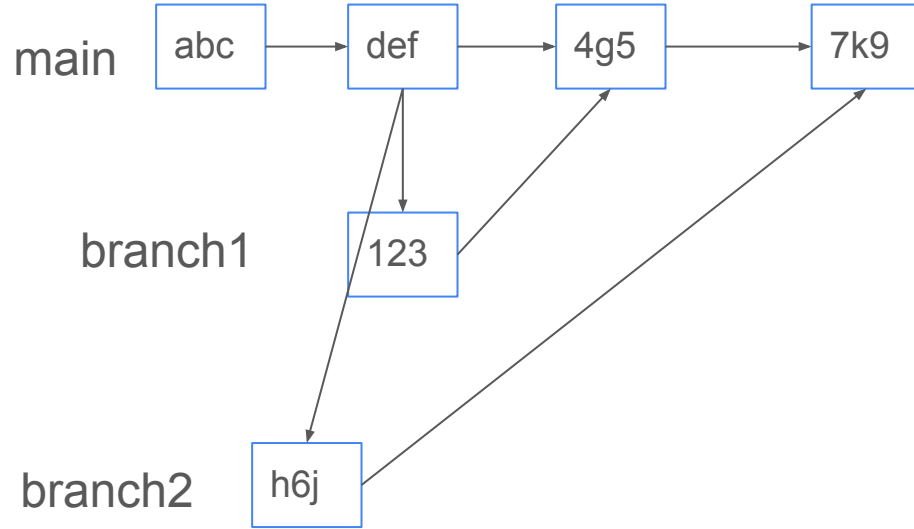
Git

Which of the following will cause a merge conflict?

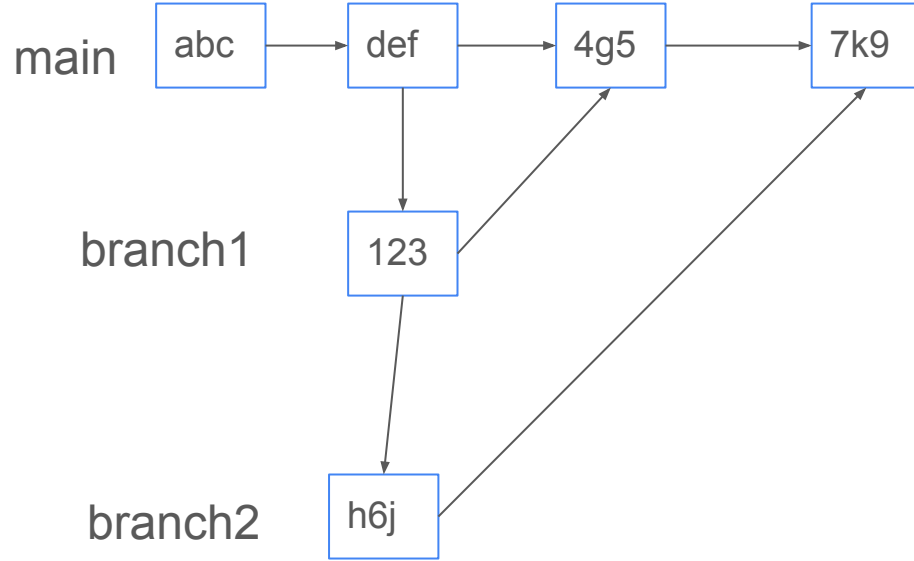
Git



Git



Git



Resolving Conflicts

Treat an action as a "change" to the server state.

Ex: User picks up a block of dirt in Minecraft

```
ServerState initial: int gridValue = FLOATING_BLOCK_DIRT
```

```
UserAction userAction: () -> pickUpDirt();
```

```
ServerState newState = applyUserAction(initial, userAction);
```

```
newState: int gridValue = EMPTY
```

Resolving Conflicts

Treat an action as a "change" to the server state.

If the order in which changes are applied doesn't matter, there's no conflict:

```
applyUserAction(applyUserAction(initial, userAction1),  
userAction2)
```

==

```
applyUserAction(applyUserAction(initial, userAction2),  
userAction1)
```

Git

More semantic knowledge $\Rightarrow$ better automated conflict resolution

Git knows about:

- files
- lines of text

It does not know:

- how to compile Java code
- whether or not whitespace matters